

signus 코드 원리 학습 문서

블라인드 신호 복조기 — 코드를 타이핑하기 전에 원리부터

numpy + scipy 전용 · 에어갭 배포용

핵심 코어 ~1,850줄 · 웹 UI ~1,070줄

2026-07 · 최종 QA 통과본

이 문서의 목표

이 문서는 signus를 손으로 타이핑해 에어갭 머신에 옮기기 전에, 코드가 무엇을 왜 그렇게 하는지 이해하도록 돕습니다. DSP를 처음 접해도 따라올 수 있게 "개념 → 원리 → 코드 위치" 순서로 설명하고, 각 설계 결정의 이유 (왜?)를 함께 답니다.

목차

1. signus는 무엇인가 — 블라인드 복조라는 문제
2. 큰 그림: 신호 처리 파이프라인
3. 기초 개념 5가지 (IQ·반송파·심볼레이트·변조·별자리)
4. 단계별 원리와 코드 — 선형(PSK/QAM) 경로
5. FSK 경로 — 주파수로 말하는 신호
6. 프리앰블 동기 — 짧은 패킷을 살리는 법
7. 어렵게 얻은 교훈 (hard-won facts)
8. 코드 읽는 순서 & 모듈 지도
9. 부록: 용어집

1. signus는 무엇인가 — 블라인드 복조라는 문제

무전기·SDR로 잡은 미지의 디지털 신호 파일(IQ 또는 PCM)을 던져 넣으면, signus는 사람이 아무 정보도 주지 않아도 스스로 **제원(반송파 주파수, 심볼레이트, 변조 방식)**을 추정하고 복조해서, 시간 순서대로 움직이는 **별자리(constellation) 애니메이션**과 **lock 점수**, 그리고 복원된 비트열을 보여줍니다.

"블라인드(blind)"의 의미. 보통의 수신기는 "이건 100 kbaud QPSK, 반송파 12 kHz" 처럼 규격을 미리 알고 있습니다. signus는 그걸 **모른 채** 신호 자체의 통계적 성질만으로 역추정합니다. 파일 이름에는 읽기에 필요한 정보(fs, 샘플 타입)만 있고, 정답(변조 방식 등)은 분석기가 **절대 읽지 않는** <파일>.json 사이드카에만 들어 있습니다.

왜 어려운가? 반송파를 모르면 별자리가 빙글빙글 돌고, 심볼레이트를 모르면 어느 순간을 "심볼"로 볼지 모르며, 변조를 모르면 점이 4개인지 16개인지도 모릅니다. 이 세 가지가 서로 얽혀 있어(닭이 먼저냐 달걀이 먼저냐), 한 번에 하나씩 통계로 뽑아내는 순서가 곧 파이프라인입니다.

2. 큰 그림: 신호 처리 파이프라인

선형 변조(PSK/QAM)의 처리 흐름은 다음과 같습니다. 각 상자가 하나의 함수이고, 옆에 코드 위치를 적었습니다.

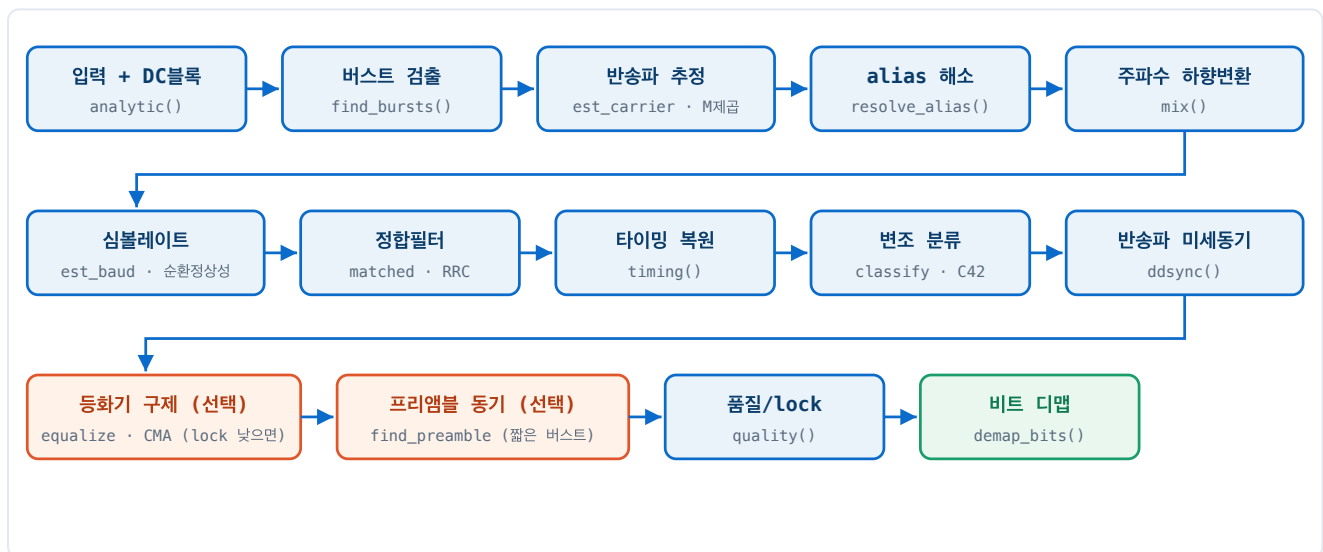


그림 1. 선형(PSK/QAM) 복조 파이프라인. 주황색은 필요할 때만 도는 "구제(rescue)" 단계. 전체는 `pipeline.analyze()` 안에 순서대로 들어 있다.

핵심 아이디어: "쉬운 것부터, 하나씩 벗겨낸다." 먼저 반송파를 없애 신호를 기저대역으로 내리고 → 심볼레이트를 찾아 그 속도로 표본화하고 → 그제서야 별자리 모양(변조)을 판단합니다. 잘 안 되면(lock이 낮으면) 등화기나 프리앰블 동기 같은 추가 도구를 **조건부로** 꺼내 씁니다.

3. 기초 개념 5가지

① IQ 샘플과 analytic 신호

복소수 샘플 $I + jQ$ 는 신호의 진폭과 위상을 동시에 담습니다. 실수(PCM) 신호는 힐베르트 변환으로 복소(analytic) 신호로 바꿉니다(`dsp.analytic`). 맨 앞에서 평균을 빼 **DC(직류) 성분**을 제거하는데, 이는 LO 누설이 모든 추정기를 오염시키기 때문입니다.

② 반송파(carrier)와 주파수 오프셋

신호가 실려 있는 중심 주파수입니다. 송·수신기 오차 때문에 남는 잔여 오프셋(CFO)을 없애지 않으면 별자리가 회전합니다. `signus`는 이를 추정해서 곱셈으로 상쇄합니다(`mix`).

③ 심볼레이트(baud)와 오버샘플링

초당 심볼 수입니다. 샘플레이트 `fs`가 baud의 10배면 "심볼당 10 샘플(`sps=10`)". `signus`는 내부에서 정확히 `sps=4`로 리샘플해 정합필터·타이밍을 겁니다(`to_sps`).

④ 변조(modulation): PSK / QAM / FSK

계열	정보를 실는 곳	예시
PSK (위상)	반송파의 위상	BPSK(2) · QPSK(4) · 8PSK(8)
QAM (진폭+위상)	진폭 과 위상	16QAM · 32QAM(십자) · 64QAM
FSK (주파수)	순시 주파수	FSK2 · FSK4 · MSK

⑤ 별자리(constellation)

복소평면 위 이상적인 심볼 점들의 배치. 복조란 결국 수신 점을 가장 가까운 이상점으로 "판정(hard decision)"하는 일입니다. 아래는 `signus`가 쓰는 이상 별자리입니다.

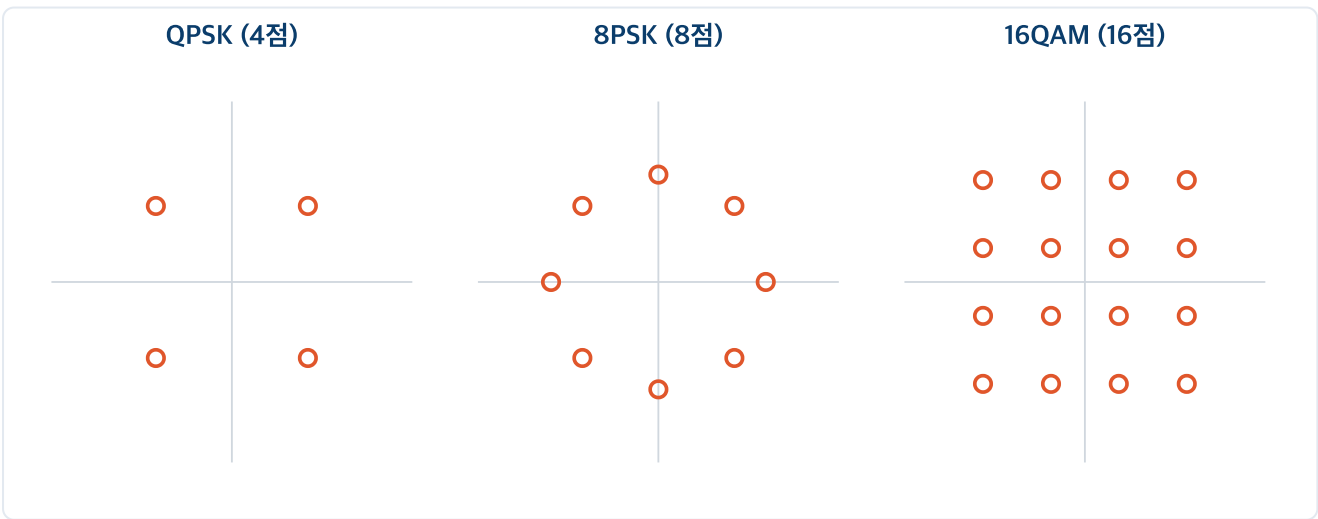


그림 2. 이상 별자리. PSK는 원 위에, QAM은 격자 위에 점이 놓인다. (`constellations.ideal_points`)

실제 수신 신호에는 잡음이 끼어서 점이 이상점 주위에 구름처럼 퍼집니다. 이 구름이 얼마나 조밀한가가 곧 **lock 점수** (0~100)입니다.



그림 3. signal가 실제로 복조한 수신 별자리(초록 점 = 수신 심볼, 주황 원 = 이상점). 왼쪽 QPSK는 4개 뭉치가 또렷 → lock 높음.

4. 단계별 원리와 코드 — 선형(PSK/QAM) 경로

4.1 버스트 검출 `dsp.find_bursts`

신호가 실제로 존재하는 구간(버스트)을 에너지로 찾습니다. 로그 전력 히스토그램을 Otsu 방법으로 잘라 잡음 바닥을 정하고, 바닥보다 6~10 dB 높은 구간을 버스트로 봅니다. 아무것도 없으면 전체를 한 버스트로 반환합니다(`return [(0, n)]`) — 그래서 뒤 단계가 절대 빈 리스트로 터지지 않습니다.

4.2 반송파 추정 — M제곱의 마법 `dsp.est_carrier`

PSK 신호는 위상이 계속 바뀌어서, 그냥 스펙트럼을 봐도 반송파 톤이 안 보입니다. 그런데 신호를 **M제곱**하면 변조가 "벗겨지면서" $4fc$ (QPSK), $8fc$ (8PSK) 위치에 뾰족한 톤이 살아납니다.

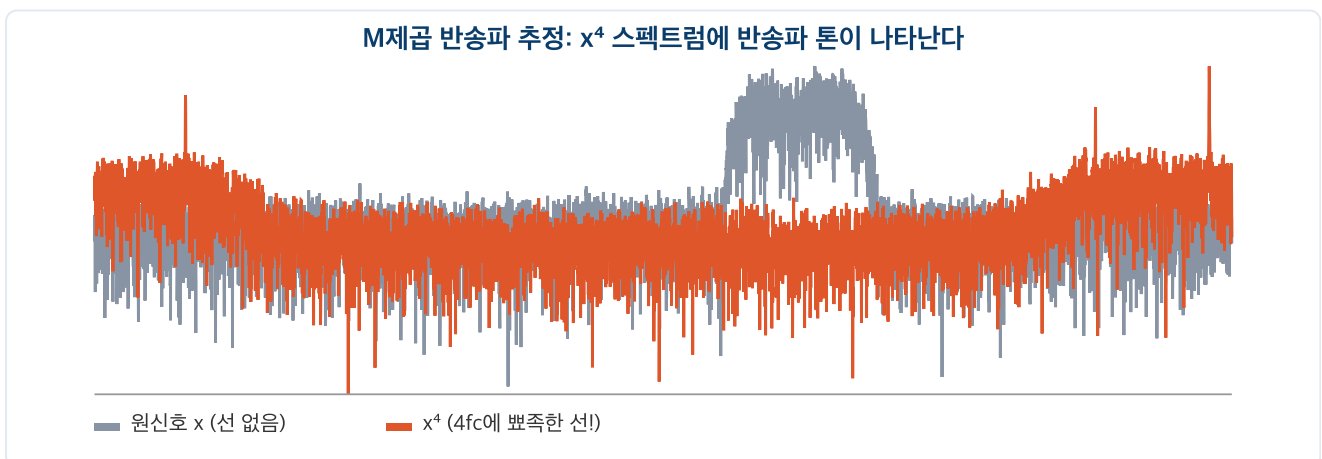


그림 4. 원신호(회색)에는 뚜렷한 반송파 선이 없지만, x^4 (주황)에는 $4 \cdot fc$ 위치에 날카로운 톤이 생긴다. 이 톤을 찾아 M으로 나누면 반송파 추정.

```
# est_carrier: p=2,4,8을 각각 시도해 가장 뽕족한(peak/mean) 스펙트럼을 고른다
for p in powers:          # (2, 4, 8)
    xp = x ** p            # M제곱 → 변조 제거
    # ... 블록 평균 FFT ...
    par = spec.max() / spec.mean() # peak-to-mean = 톤이 얼마나 뽕족한가
    if par > best_par:      # 가장 뽕족한 p가 곧 그 신호의 대칭 차수
        best_par, best_s, sym = par, spec, p
fc = f_peak / sym          # 톤 위치를 M으로 나누면 반송파
```

왜 블록을 쪼갬까? M제곱 스펙트럼은 **비간섭적으로 평균**되기 때문에, 약한 톤(높은 대칭·낮은 SNR)일수록 블록을 **적게·길게** 잡아야 톤이 잡힙니다. `blocks = clip(n/30000, 1, 8)` — 8PSK 경계에서 +2 dB를 법니다. (교훈 #1)

4.3 Alias 해소 dsp.resolve_alias

M제곱 톤은 fs를 법으로 감기기 때문에, fc는 **fs/sym의 배수만큼 모호**합니다. 후보들을 대역 전체에 깔고, 각 후보의 alias "셀"에 실제 전력이 가장 많이 든 것을 고릅니다(스펙트럼 중심보다 대역 끝에서 강건). 안 그러면 깊게 감긴 반송파가 엉뚱한 위치로 풀립니다.

4.4 심볼레이트 추정 — 순환정상성 dsp.est_baud

$|x|^2$ (포락선 제곱)에는 심볼 전이마다 생기는 규칙적 성분이 있어, 그 스펙트럼에 **baud 위치에 선(line)**이 섭니다. 이 선을 찾는 게 심볼레이트 추정입니다. 낮은 롤오프(α)에서는 선이 약해져서, 점유대역폭 사전정보로 재탐색하는 폴백이 붙어 있습니다.

4.5 정합필터 + 타이밍 dsp.matched · dsp.timing

송신 펄스(RRC)에 맞춘 정합필터로 SNR을 최대화하고(`matched`), Oerder-Meyr 방법으로 각 심볼의 **최적 표본 위치**를 찾습니다(`timing`). 블록마다 위상을 unwrap해서 클록 드리프트까지 추적합니다.

4.6 변조 분류 — C42와 진폭 링 classify.classify

대칭 차수(2→BPSK, 8→8PSK)로 갈리지 않는 4대칭 무리(QPSK/16/32/64QAM)는 두 통계로 나눕니다.

- **C42**(정규화 4차 큐물런트): 일정 진폭(PSK)이면 ~1.0, QAM이면 <0.7 → QPSK 판별.
- **진폭 링**: 진폭을 정렬해 k-평균으로 링 개수를 재고, 그 비율로 16/32/64QAM 구분. (최소자승 EVM은 금지 — 조밀한 격자가 항상 더 잘 맞아 차수를 부풀리기 때문)

4.7 등화기 구제 — 다중경로 ISI eq.equalize

다중경로(반사파)가 있으면 심볼끼리 번져(ISI) 링 판정이 어긋납니다. 위상만 도는 반송파 루프로는 못 고칩니다. 그래서 **lock이 60 미만일 때만** CMA(정진폭) 등화기로 눈을 연 뒤 **다시 분류**하고, lock이 확실히 좋아질 때만 채택합니다(아니면 오히려 엉뚱한 변조로 고정될 위험).

5. FSK 경로 — 주파수로 말하는 신호 fsk.py

FSK/MSK는 위상이 아니라 **순시 주파수**에 정보를 싣습니다. 그래서 선형 경로를 타지 않고, **정진폭 게이트**(`fsk_gate` : 포락선이 거의 일정 + 순시주파수가 다봉이면 FSK)를 통과한 버스트만 주파수 판별기로 복조합니다.

FSK 순시주파수: 두 개의 톤(주파수 준위) 사이를 오간다

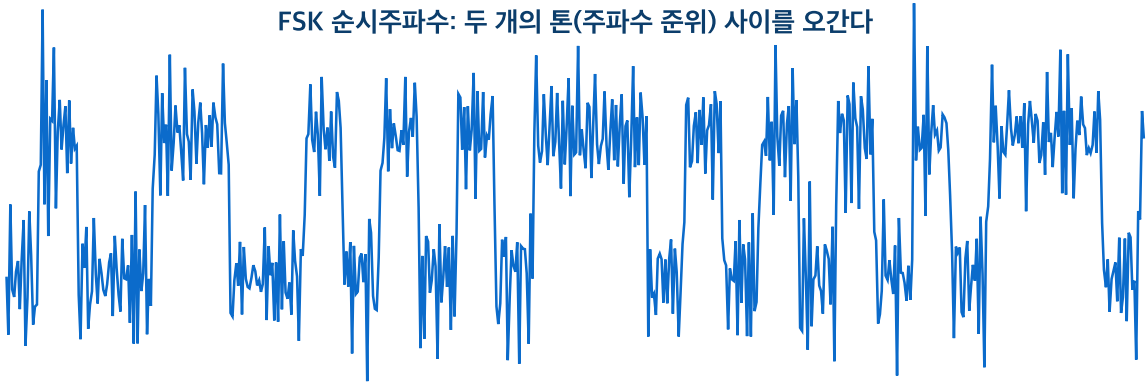


그림 5. FSK 순시주파수. 신호가 두 주파수 준위(톤) 사이를 오가며 비트를 싣는다. 준위 사이 간격 ÷ baud = 변조지수 h.

왜 별도 경로인가? FSK는 별자리 개념 자체가 없습니다(위상은 연속으로 회전). 억지로 QAM 격자에 맞추면 의미 없는 심볼이 나옵니다. 그래서 "이건 FSK다"를 먼저 판정하고 다른 파이프라인으로 보냅니다. 반대로, PSK/QAM를 FSK로 **잘못 보내면** 확신에 찬 오복조가 되므로 게이트 보정이 매우 중요합니다.

6. 프리앰블 동기 — 짧은 패킷을 살리는 법 sync.py

아주 짧은 버스트(수십~수백 심볼)는 M제곱 추정기 평균할 심볼이 모자라 실패합니다. 하지만 실제 패킷 신호는 앞에 **프리앰블(짧은 블록을 여러 번 반복)**을 담니다. 이 반복은 내용을 몰라도 **지연-상관(Schmidl-Cox)**으로 검출됩니다: 주기 P만큼 떨어진 두 창이 겹치면 상관값 $M(d)$ 가 ~1로 치솟습니다. 그 상관 위상에서 **CFO(반송파 오프셋)**를 소수의 심볼만으로 정확히 뽑습니다.

이 기능의 핵심 위험 — 회전 alias. M-PSK에서 반송파가 baud/L만큼 틀어지면 별자리 전체가 심볼마다 한 칸씩 회전합니다. 그러면 **눈은 여전히 lock되지만 비트는 어긋난**, 확신에 찬 오복조가 됩니다. 이건 블라인드로는 원리상 완전히 구분 불가능하므로, signus는 (1) 높은 lock 하한과 (2) 승자 반송파가 거친 반송파 추정치(anchor) 근처에 있을 것 — 두 조건을 동시에 걸어 **정밀도 우선**으로 회전 alias를 거절합니다(모호하면 정직하게 낮은 lock으로 남김).

7. 어렵게 얻은 교훈 (hard-won facts)

아래는 "다시 유도하지 말 것"으로 코드 주석에 박아 둔 핵심 결정들입니다. 타이핑할 때 주석을 지우지 마세요 — 계산식이 아니라 **왜 그 값인가**를 담고 있습니다.

주제	교훈
M제곱 평균	비간섭 평균이라 약한 톤은 블록을 적게·길게 . <code>blocks=clip(n/30000,1,8)</code> .
32QAM 십자	5개 진폭 링 → $s_3 \cdot s_9 / s_5^2 > 1.07$ 로 판별. 최소자승 EVM 금지(차수 편향).
다중경로	CMA는 정진폭만 보므로 등화 후 재분류 . <code>lock이 >3 그리고 ≥50일</code> 때만 채택.
$\pi/4$ -DQPSK	4제곱이 항상 π 라 반송파가 <code>baud/8</code> 흡수 → QPSK로 보임. 그래도 차동 디맵이면 <code>BER=0</code> .
OQPSK	C42·링 모든 특징이 QAM과 충돌 → 블라인드 클래스로 출하 금지 (생성 전용).
회전 alias	<code>lock</code> 만으론 못 가림. 절대 반송파 <code>anchor</code> 근접 조건을 함께 걸어야 함.

8. 코드 읽는 순서 & 모듈 지도

처음 읽는다면 이 순서를 권합니다: ① `constellations.py` (별자리·라벨의 사전) → ② `dsp.py` (추정기들) → ③ `classify.py` (판정) → ④ `pipeline.py` (`analyze()`) 가 이 모듈을 순서대로 엮는 지휘부) → ⑤ 나머지 (`fsk·sync·eq·survey`).

모듈	역할	줄수
<code>constellations.py</code>	별자리·Gray 라벨·차동표·FSK 준위 (공유 사전)	~100
<code>dsp.py</code>	수신 DSP 단계 전부 (반송파·baud·정합·타이밍·등화 입력)	~293
<code>classify.py</code>	변조 분류·회전 정렬·lock/SNR 품질	~123
<code>pipeline.py</code>	지휘부: <code>analyze()/survey()</code> — 모든 단계를 엮음	~440
<code>fsk.py</code>	FSK 게이트 + 주파수 판별기 복조	~135
<code>sync.py</code>	블라인드 반복·프리앰블 동기 (Schmidl-Cox)	~112
<code>eq.py</code> / <code>_accel.py</code>	CMA 등화기 / (선택) numba 가속 커널	~44 / ~100
<code>detect·channelize·chirp·trriage</code>	광대역 서베이 프런트엔드 (신호 탐지·채널화·처프)	—
<code>sigio.py</code> / <code>server.py</code>	파일 입출력(포맷·엔디안·SigMF) / 웹 UI 서버	~139 / ~89

anti-shared-bug 규칙: 생성기(`gen.py`)와 수신기(`dsp.py`)는 DSP 코드를 **공유하지 않습니다**(별자리 참조 데이터만 공유). 그래서 수신기 버그를 쌓듯이 생성기 버그가 가려주는 일이 없습니다. 테스트의 신뢰성이 여기서 나옵니다.

9. 부록: 용어집

IQ	동위상(I)·직교(Q) 성분으로 된 복소 표본
fs / baud	샘플레이트 / 초당 심볼 수. fs/baud = 심볼당 샘플(sps)
fc / CFO	반송파 주파수 / 잔여 반송파 오프셋
롤오프 α	RRC 펄스의 대역 여유(0=날카로움, 1=완만)
lock	별자리가 얼마나 조밀·정렬됐나 (0~100). MER×점유율 기반
EVM / MER	오차 벡터 크기 / 변조 오차비 (품질 지표)
ISI	심볼간 간섭 (다중경로가 유발) → 등화기로 보정
alias	샘플링·M제곱으로 주파수가 접혀 생기는 모호성
Schmidl-Cox	반복 프리앰블을 지연-상관으로 검출하는 동기 기법